Spyder IDE Basics for CIVL6410

Jiwon Kim, The University of Queensland

2026

Contents

Spyder IDE Basics for CIVL6410

1. The Spyder Layout

When you open Spyder you will see three main panels:

Panel	Default location	Purpose
Editor	Left / centre	Write and edit `.py` files
IPython Console	Bottom right	Run code interactively, see output
Viewing panel	Top right	Multiple tabs: Files, Variable Explorer, Plots, and more

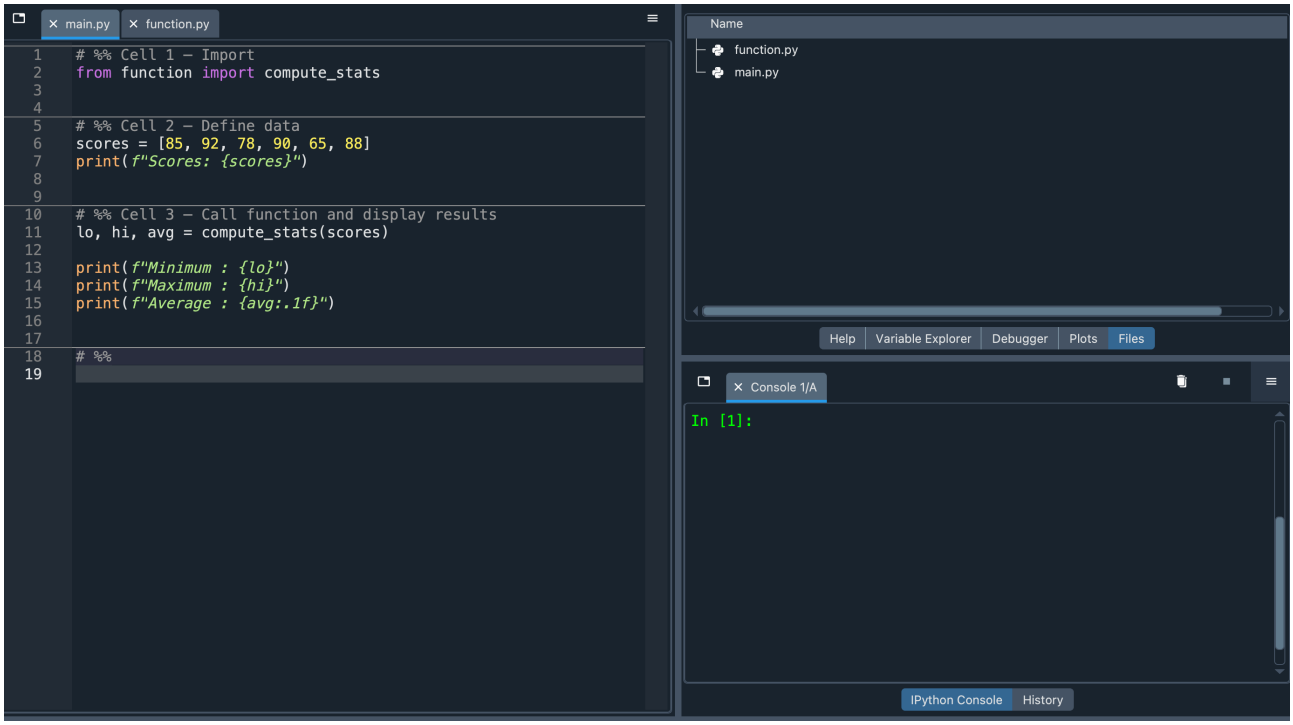

Figure 1: Spyder window showing the three main panels: Editor (left), IPython Console (bottom right), and the viewing panel with Files, Variable Explorer, and other tabs (top right).

Key points

- The **Editor** is where you write scripts. Code does not run automatically when you type it.
- The **IPython Console** is a live Python session. Variables you create persist until you restart the kernel.
- The **viewing panel** (top right) has multiple tabs. The two most useful are **Files** (browse and open files) and **Variable Explorer** (inspect every variable currently in memory — name, type, size, and value).

2. Running Code — Four Ways

Fig. 2 shows the Spyder toolbar. Buttons (1)–(4) control how a script is run. Buttons (5)–(7) are debug controls that let you pause execution and step through code line by line — these are covered in

Section 4.

Figure 2: Spyder toolbar. (1) Run File (F5), (2) Run Cell (Ctrl+Enter), (3) Run Cell and Advance (Shift+Enter), (4) Run Selection (F9). Buttons (5)–(7) are debugging controls, covered in Section 4.

No. in Fig. 2	Action	Shortcut	What it does
(1)	Run file	`F5`	Runs the entire script from top to bottom
(2)	Run cell	`Ctrl+Enter`	Runs only the current cell, cursor stays
(3)	Run cell and advance	`Shift+Enter`	Runs the current cell, moves to the next
(4)	Run selection	`F9`	Runs the highlighted text, or the current line if nothing is selected

Key points

- **Run file (F5)** executes the entire script from top to bottom — use it when the full workflow is complete and you want a clean end-to-end run.
- **Run cell (Ctrl+Enter)** is useful during development — you can test one section at a time without re-running everything.
- **Run selection (F9)** is the most flexible — highlight any snippet, or just place the cursor on a single line, and press `F9` to run only that. Useful for quickly testing one expression or inspecting a variable without defining a cell.
- If you get a `NameError`, it usually means you skipped a cell (or line) that defines a variable. Run from the top in order.

3. Cell-Based Coding with # %%

A **cell** is a block of code separated by a `# %%` comment. Spyder highlights the active cell with a yellow background.

To demonstrate cell-based running (Section 3) and debugging (Section 4), we will create two small demo files: `function.py`, which defines a function, and `main.py`, which calls it cell by cell. These same files are used throughout the rest of this tutorial.

3.1. Create the demo files

Create a new folder (e.g. `demo/`) and place both files inside it. Then set `demo/` as the working directory so that Spyder can find both files when running or importing.

`function.py` — type this in full:

```
def compute_stats(scores):
    """Return the minimum, maximum, and average of a list of numbers.
```

```
    Parameters
    ----------
    scores : list   a list of numeric values (e.g. exam scores)

    Returns
    -------
    minimum : float  the smallest value
    maximum : float  the largest value
    average : float  the mean value
    """
    total   = 0
    minimum = scores[0]
    maximum = scores[0]

    for s in scores:
        total += s
        if s < minimum:
            minimum = s
        if s > maximum:
            maximum = s

    average = total / len(scores)
    return minimum, maximum, average
```

---

main.py — type this in full:

```
# %% Cell 1 - Import
from function import compute_stats

# %% Cell 2 - Define data
scores = [85, 92, 78, 90, 65, 88]
print(f'Scores: {scores}')

# %% Cell 3 - Call function and display results
lo, hi, avg = compute_stats(scores)

print(f'Minimum : {lo}')
print(f'Maximum : {hi}')
print(f'Average : {avg:.1f}')
```

Save both files in the `demo/` folder.

3.2. Run the cells

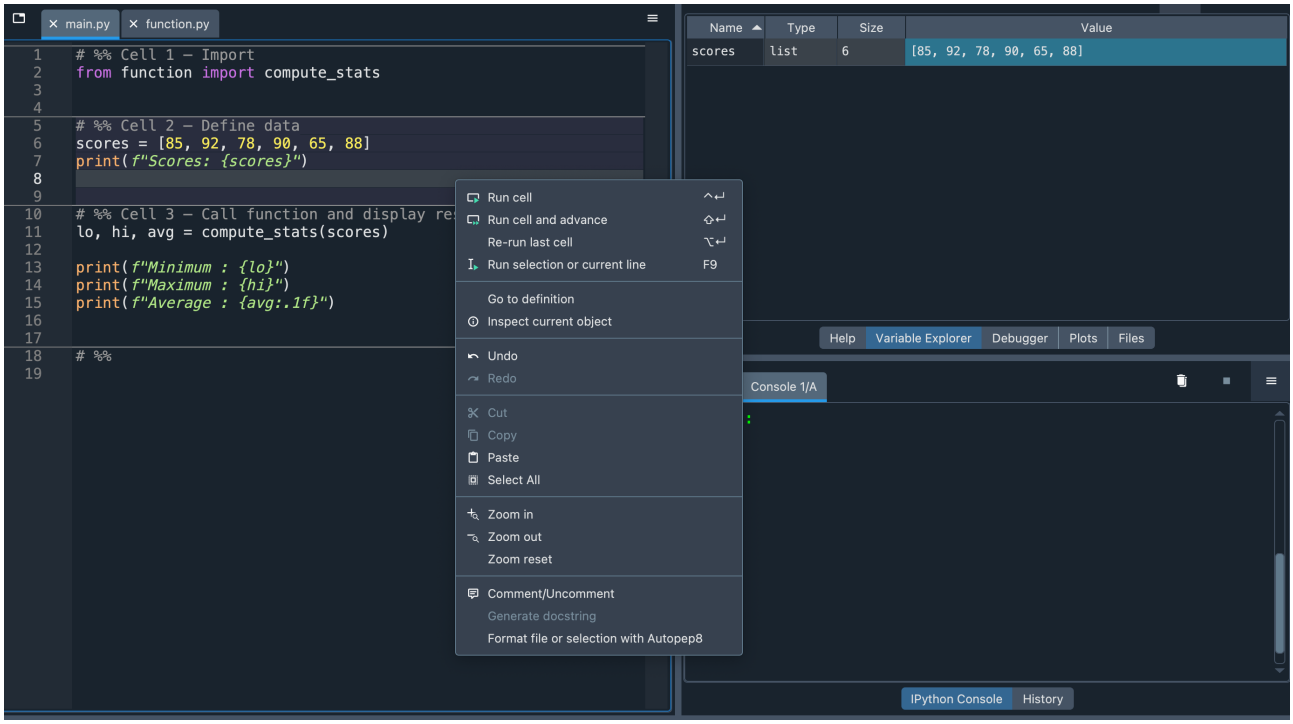

Figure 3: main.py open in the Editor with `# %%` cell separators. The active cell is highlighted in shaded color.

Try it yourself

1. Open `main.py` in Spyder (you just created it).
2. Click anywhere inside **Cell 1** and press `Ctrl+Enter`. Check the IPython Console — no output, but `compute_stats` is now imported.
3. Click inside **Cell 2** and press `Ctrl+Enter`. The Variable Explorer should now show `scores`.
4. Click inside **Cell 3** and press `Ctrl+Enter`. You should see the min, max, and average printed.

Key points

- `# %%` is just a comment — it works in any Python file, but Spyder uses it to define cell boundaries.
- You can give cells a name by writing text after `# %%`, e.g. `# %% Cell 1 - Import`.
- Cells are independent only in the sense that you choose when to run them — variables from earlier cells are still in memory.

4. Debugging Across Files

What is debugging?

Running a script (e.g. `F5`) executes every line from top to bottom and gives you the final output. That is sufficient when the code works as expected. **Debugging** lets you pause execution at a chosen line — called a *breakpoint* — and then advance one line at a time, inspecting every variable at each step. Use debugging when:

- a function returns a wrong value and you cannot tell *why* from the output alone,
- you want to understand how a complex piece of logic actually executes, or

- an error occurs deep inside a function which is hard to trace.

Debugging is particularly useful when the function you want to inspect is **defined in a separate file** from the script that calls it. Spyder lets you set breakpoints in any open file — including files your script imports from — and then step into those functions as if the code were all in one place.

The debug toolbar

As you saw in Fig. 2, the Spyder toolbar normally shows run controls (Fig. 2 buttons (1)–(4)) and debug shortcut (Fig. 2 buttons (5)–(7)). To start debugging, click button **(5) Debug file** in Fig. 2. Once you do, the toolbar expands into the full **debug toolbar** (Fig. 4), revealing additional controls — step over, step into, step return, continue, and stop — that let you walk through code one line at a time. The debug toolbar stays visible until you press Stop.

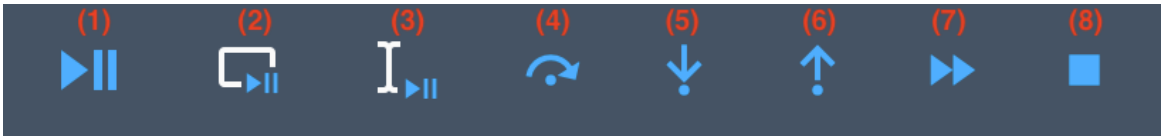

Figure 4: Debug toolbar. (1) Debug file, (2) Debug cell, (3) Debug selection, (4) Step over, (5) Step into, (6) Step return, (7) Continue, (8) Stop.

No. in Fig. 4	Button	Action
(1)	**Debug file**	Start debugging the entire file from the top
(2)	**Debug cell**	Start debugging the current cell only
(3)	**Debug selection**	Start debugging the selected code only
(4)	**Step** (step over)	Execute the current line without entering functions
(5)	**Step into**	Step into the function on the current line
(6)	**Step return**	Run until the current function returns
(7)	**Continue**	Run until the next breakpoint
(8)	**Stop**	Exit debug mode and return to normal execution

Demo: stepping into a function in another file

Scenario: `main.py` calls `compute_stats()` which is defined in `function.py`. You want to step through the loop inside `compute_stats()` to watch how `minimum`, `maximum`, and `total` are updated with each iteration.

4.1. Open both files

Open both `function.py` and `main.py` in the Editor. Both files will appear as tabs the Editor.

4.2. Set a breakpoint in `function.py`

Click on the line number to the left of the `for s in scores:` line inside `function.py`. A red dot (breakpoint) will appear, as shown in #fig:breakpoint.

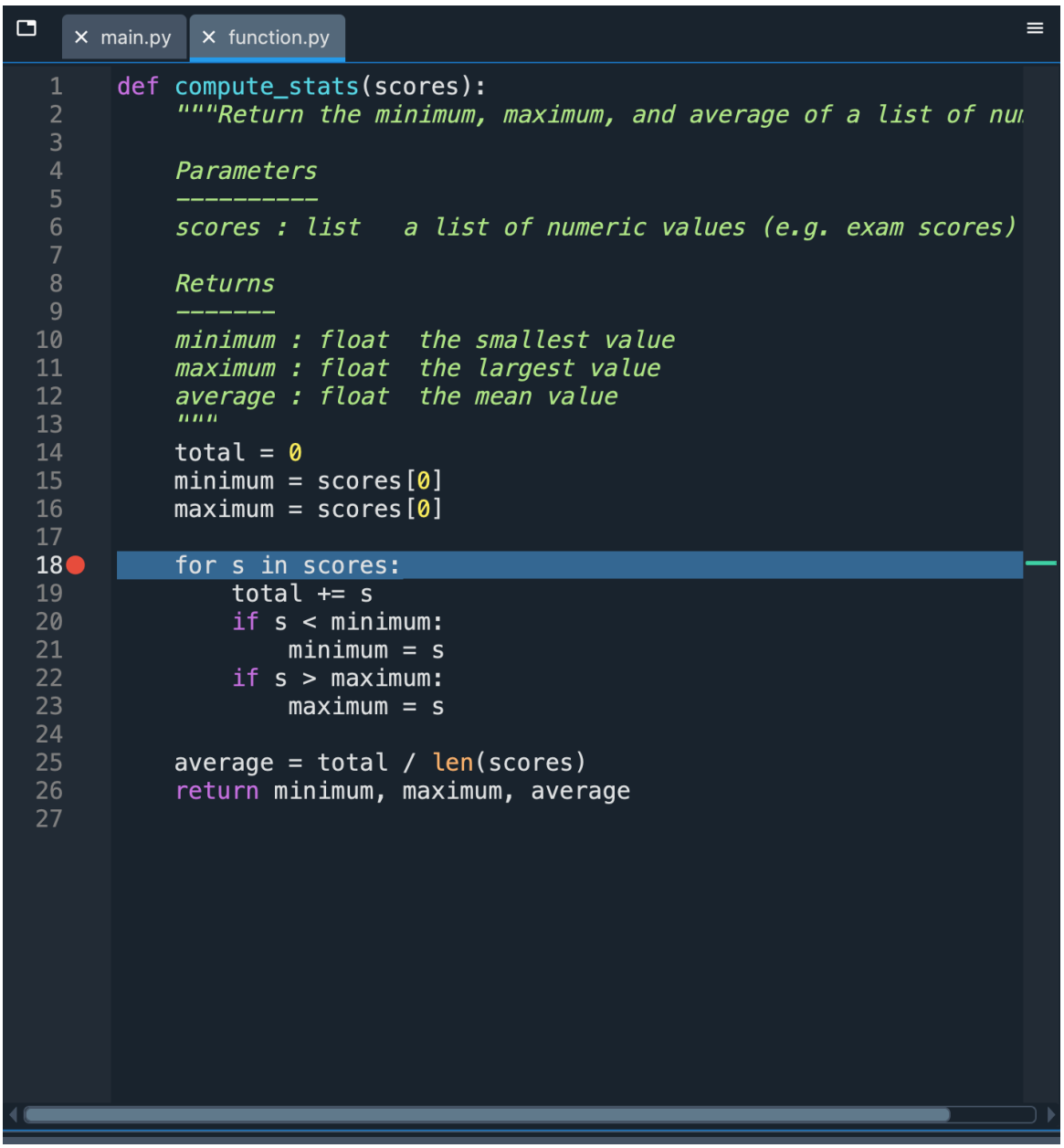

Figure 5: function.py with a red breakpoint dot on the `for s in scores:` line.

4.3. Run `main.py` in debug mode

Before starting, enable a helpful debugger setting: go to `Preferences → Debugger` and tick **"Stop debugging on first line of files without breakpoints"**. With this on, Spyder pauses at the very first line of your script when you enter debug mode, giving you a chance to check the state before anything runs. You can then press **Continue** to advance to your breakpoint.

Switch to the `main.py` tab. Click button **(5) Debug file** in Fig. 2. The toolbar will expand into the debug toolbar (Fig. 4).

Spyder pauses immediately at the first line of `main.py` (Fig. 6). The blue arrow shows where execution is currently stopped.

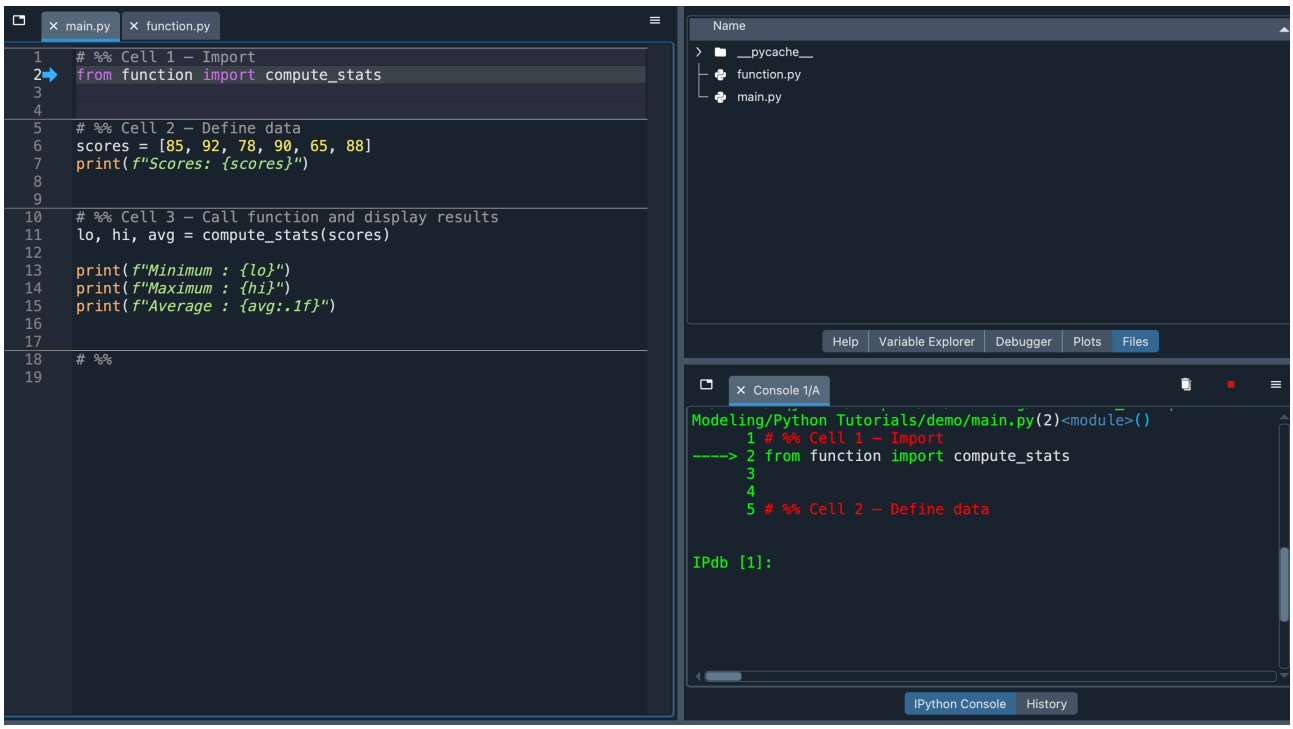

Figure 6: Debug mode has started — Spyder has paused at the first line of `main.py`.

Notice that the IPython Console prompt has changed from `In [n]:` to `ipdb>`. This means you are now inside the **IPdb** debugger. You can type any Python expression at this prompt to inspect or test values at the current point of execution just as you would in the normal console.

Press **Continue** (button (7) in Fig. 4) to run forward until the breakpoint you set in `function.py` is hit. Spyder will jump into `function.py` and pause at the `for s in scores:` line (Fig. 7).

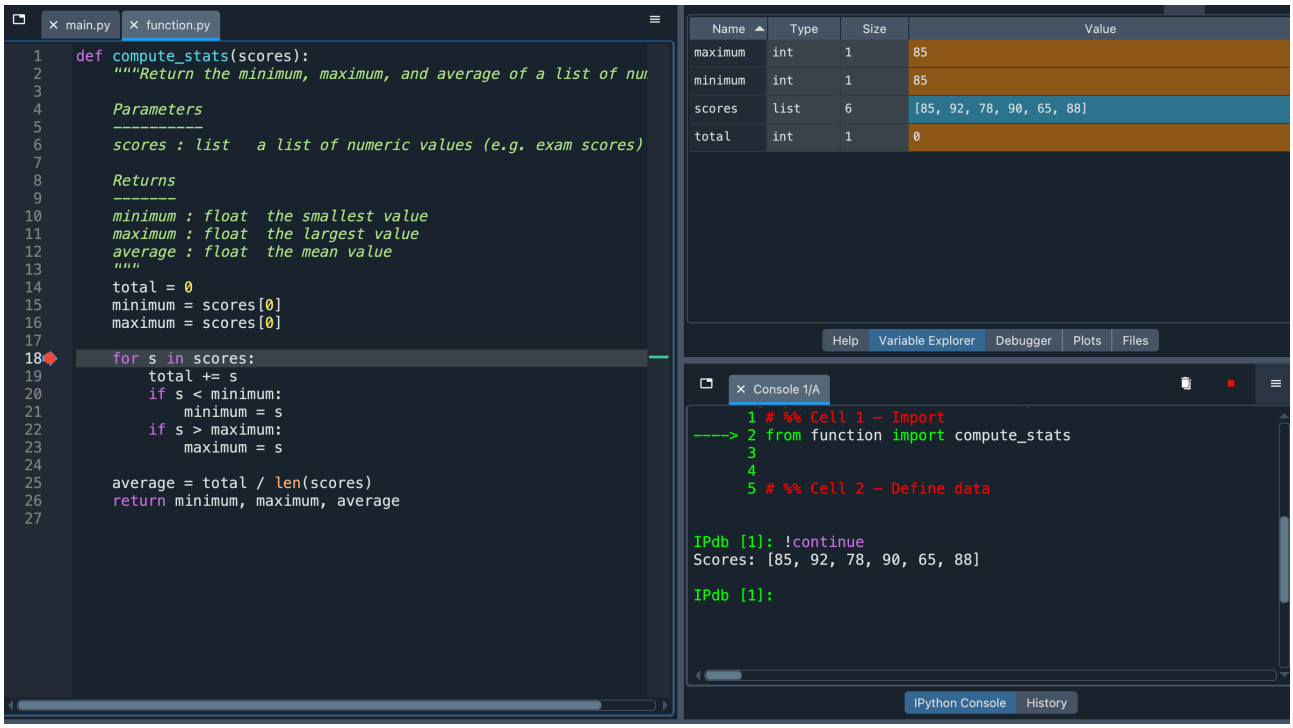

Figure 7: Spyder paused at the breakpoint inside `function.py`. The blue arrow marks the current line, which is at the breakpoint.

4.4. Step through the code

Press **Step** (button (4) in Fig. 4) repeatedly to advance through the loop one step at a time. After each step, look at the **Variable Explorer** — watch `total`, `minimum`, and `maximum` update with each line executed.

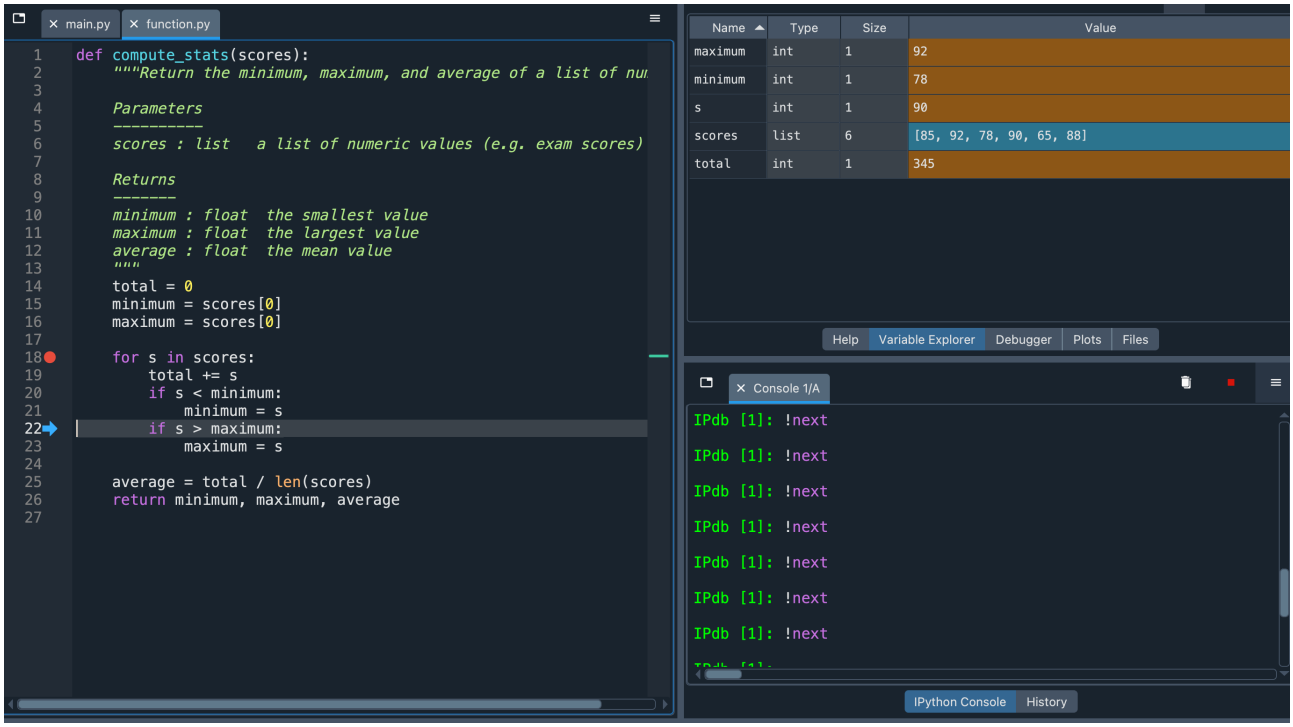

Figure 8: Variable Explorer during debug, showing `s`, `total`, `minimum`, and `maximum` updating as the loop steps through each iteration.

4.5. Stop debugging

When you are done, click **Stop** (button (8) in Fig. 4) to exit debug mode. The `ipdb>` prompt in the console will return to the normal `In [n]:` prompt, and the debug toolbar will be replaced by the standard run toolbar.

Note that once you stop, the **Variable Explorer will no longer show the variables** that were in scope during debugging — they only exist inside the function's local scope while the debugger is active.